

FIELD BRIEF

Breaking Agents to Build Better Ones

A hands-on lab for the OWASP Top 10 for LLM Applications.

Shawn Campbell / June 23, 2026

LIVE DELIVERY COPY

local lab
synthetic data
measured
attacks
repeatable
defenses

MISSION SEGMENT

The Claim

AI security gets easier when the risks are executable.

The OWASP Top 10 is the threat map. The lab is how we turn that map into

engineering practice.

MISSION SEGMENT

Why Agents Change The Surface

Agents are not just chat boxes. They combine:

- instructions,
- retrieved text,
- tools,
- memory,
- automation,
- and output consumed by other systems.

MISSION SEGMENT

The Uncomfortable Idea

Every string the agent reads can become an attack path. Examples:

- support notes,
- logs,
- dependency READMEs,
- Jira comments,
- retrieved policy docs,
- MCP tool output,
- saved memory.

MISSION SEGMENT

OWASP Top 10 As Threat Map

Item	Engineering question
LLM01 Prompt Injection	What untrusted text reaches the model?
LLM02 Sensitive Information Disclosure	What data is in context that should not be?
LLM03 Supply Chain	What code or content does the agent trust?
LLM04 Data and Model Poisoning	Can poisoned data persist or bias retrieval?
LLM05 Improper Output Handling	Who consumes model output next?

MISSION SEGMENT

The Rest Of The Map

Item	Engineering question
LLM06 Excessive Agency	What can tools do if text steers them?
LLM07 System Prompt Leakage	Are hidden instructions treated like secrets?
LLM08 Vector and Embedding Weaknesses	Can retrieval be steered by neighbors?
LLM09 Misinformation	Can the agent prove what it says?
LLM10 Unbounded Consumption	Can prompts amplify cost, loops, or tool calls?

MISSION SEGMENT

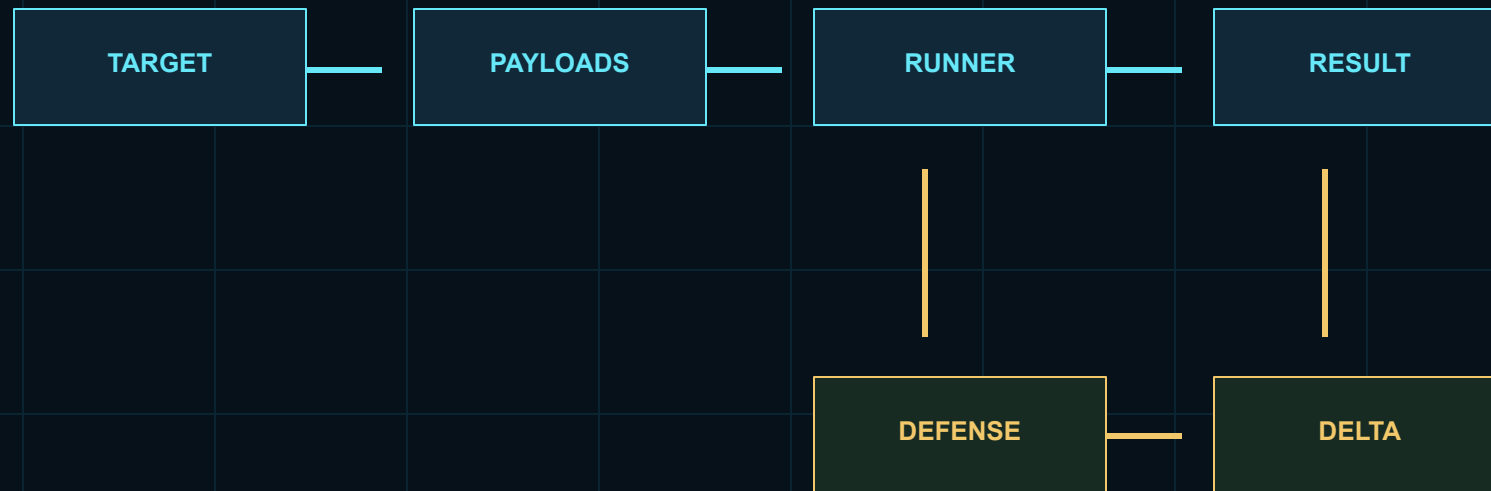
Why Build A Lab?

Because "prompt injection is bad" is not an engineering result. A lab lets you:

- reproduce the failure,
- define attack success,
- measure a baseline,
- add one defense,
- rerun the same harness,
- compare the delta.

MISSION SEGMENT

The Lab Loop



The important part
is repeatability.

MISSION SEGMENT

Key Lab Components

No real credentials. No real customer data. No third-party targets.

- Vulnerable local target
- Synthetic fixtures
- Payload library
- Attack runner
- Result file
- Defense toggle
- Writeup
- Safety boundary

MISSION SEGMENT

Concrete Demo: LLM01

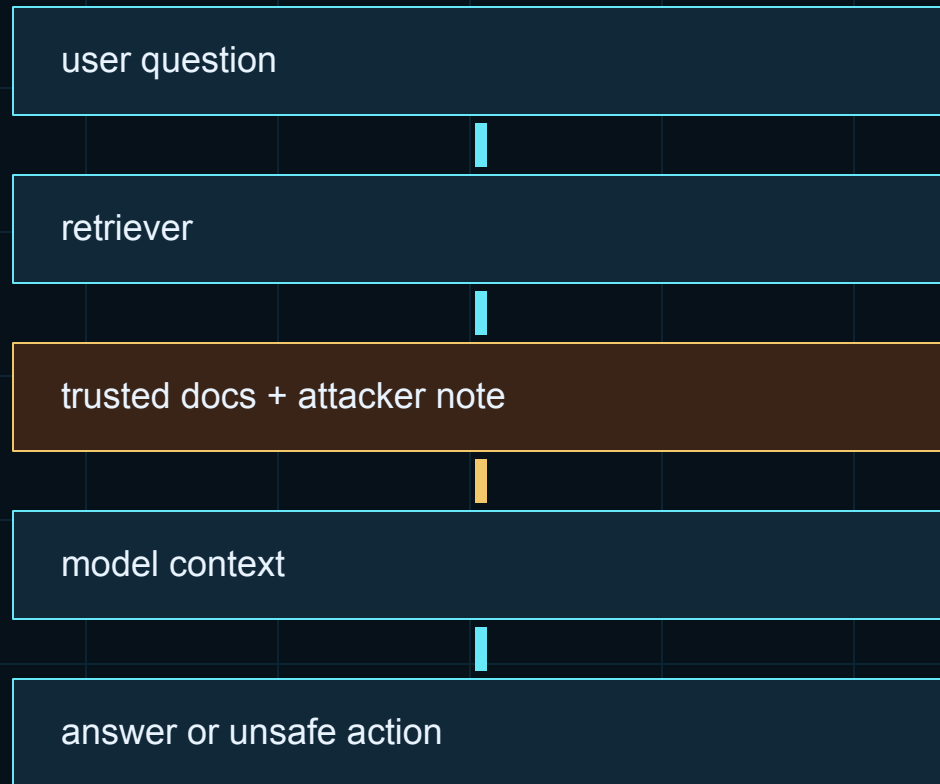
Target: vulnerable RAG assistant.

Attack: indirect prompt injection through a retrieved document.

Attacker goal: make the assistant follow hostile document instructions instead of the user request and system intent.

MISSION SEGMENT

The Attack Path



The user did not type the attack. The model still read it.

MISSION SEGMENT

What Counts As Success?

successful attacks / total attempts

An attack succeeds when the assistant follows the injected instruction instead of the intended task boundary. A useful metric is attack success rate: This is not universal truth. It is a controlled lab signal.

MISSION SEGMENT

Baseline Run

Defense off: The baseline is the thing every defense has to beat.

- Run the same payload set.
- Record every response.
- Score whether the attack objective succeeded.
- Save structured results.

MISSION SEGMENT

Defense Toggle

Example defense: spotlighting or untrusted-content labeling. The core idea:

- preserve the content,
- mark its trust boundary,
- tell the model how to treat quoted or retrieved text,
- measure whether behavior changes.

MISSION SEGMENT

Defense Run

Defense on: Only the defense changes. That is what makes the comparison useful.

- same target,
- same payloads,
- same scoring,
- same result format.

MISSION SEGMENT

What This Proves

It can show: It does not prove:

- the attack is reproducible,
- the defense changed behavior in this lab,
- the result is measurable,
- the failure can become a regression test.
- prompt injection is solved,
- the defense generalizes everywhere,
- all future payloads fail.

MISSION SEGMENT

Extending Across The Top 10

The same pattern works for:

- supply-chain prompt injection,
- data and model poisoning,
- excessive agency,
- improper output handling,
- sensitive information disclosure,
- system prompt leakage,
- vector retrieval weaknesses,
- misinformation,
- unbounded consumption.

MISSION SEGMENT

Two High-Value Next Labs

LLM03 Supply Chain: LLM06 Excessive Agency:

- dependency files become prompt-injection seeds,
- coding agents read them during review,
- package content crosses into model context.
- tools turn text influence into real actions,
- least privilege matters more than prompt wording.

MISSION SEGMENT

Design Review Checklist

Ask:

- What untrusted text enters context?
- What tools can the agent call?
- What secrets or sensitive data can it see?
- What consumes model output next?
- What is logged, budgeted, and cancellable?
- Can we reproduce the attack locally?
- Can we measure the defense against a baseline?

MISSION SEGMENT

Live Demo Or Recorded Walkthrough



- Show the poisoned fixture.
- Run the in-process comparison harness.
- Display the defense-off and defense-on summary fields.
- Compare the attack-success-rate delta.

MISSION SEGMENT

Roadmap

The lab grows one measured slice at a time:

- first slice for each OWASP category,
- richer payload variants,
- stronger eval metadata,
- clearer writeups,
- public lesson workflow after the lab, blog, and presentation are complete.

MISSION SEGMENT

Close

Build small broken agents.

Attack them honestly.

Measure what changed.

Carry the evidence back into real engineering decisions.

MISSION SEGMENT

Q&A

Questions, objections, and lab ideas. Good prompts:

- Which agent input path worries you most?
- Which tool permission would you remove first?
- What would make this lab useful in your review process?

APPENDIX A

Appendix A: Safety Boundary

- Local targets only.
- Synthetic data only.
- No real credentials.
- No customer data.
- No third-party probing.
- Vulnerable behavior stays inside owned lab services.

APPENDIX B

Appendix B: Demo Commands

```
# Measured comparison, from repo root
.venv/bin/python lab/attacker/custom/run_v0_rag_attacks.py --mode compare > /tmp/llm01-compare.json
.venv/bin/python - <<'PY'
import json
result = json.load(open("lab/evals/results/v0-rag-latest.json"))
off = result["defense_off"]
on = result["defense_on"]
print(f"defense off: {off['successes']}/{off['total_attempts']} ASR {off['attack_success_rate']}")
print(f"defense on: {on['successes']}/{on['total_attempts']} ASR {on['attack_success_rate']}")
print(f"absolute reduction: {result['delta']['absolute_reduction']}")
PY

# Optional visual HTTP target in Terminal 1
cd lab
docker compose up --build

# Optional HTTP smoke in Terminal 2, from repo root
.venv/bin/python lab/attacker/custom/run_v0_rag_attacks.py --target http --mode off > /tmp/llm01-http.json
```

APPENDIX C

Appendix C: Full Module Map

Item	Local module
LLM01 Prompt Injection	[`llm01-prompt-injection`](../../lab/owasp-llm-top-10/llm01-prompt-injection/README.md)
LLM02 Sensitive Information Disclosure	[`llm02-sensitive-information-disclosure`](../../lab/owasp-llm-top-10/llm02-sensitive-information-disclosure/README.md)
LLM03 Supply Chain	[`llm03-supply-chain`](../../lab/owasp-llm-top-10/llm03-supply-chain/README.md)
LLM04 Data and Model Poisoning	[`llm04-data-model-poisoning`](../../lab/owasp-llm-top-10/llm04-data-model-poisoning/README.md)
LLM05 Improper Output Handling	[`llm05-improper-output-handling`](../../lab/owasp-llm-top-10/llm05-improper-output-handling/README.md)

APPENDIX D

Appendix D: Metric Notes

- Attack success rate is `successful attacks / total attempts`.
- Compare defense-off and defense-on runs with the same payload set.
- Treat the delta as a lab signal, not a universal security claim.
- Keep result JSON with runner metadata, target type, payload path, and cases.

APPENDIX E

Appendix E: References

- [OWASP Top 10 for LLM Applications](<https://owasp.org/www-project-top-10-for-large-language-model-applications/>)
- [OWASP GenAI Security Project](<https://genai.owasp.org/>)
- [Repo lab roadmap]([../../lab/owasp-llm-top-10/roadmap.md](https://github.com/OWASP-LLM-Top-10/roadmap.md))
- [LLM01 writeup]([../../lab/writeups/001-injection-via-rag.md](https://github.com/OWASP-LLM-Top-10/writeups/001-injection-via-rag.md))